

Program Analysis 2025-26

Lecture #7

More about IL and Necessary Conditions



UNIVERSITÀ
DI PISA

More about IL: Soundness, Completeness

Yet another perspective

from $\llbracket c \rrbracket \subseteq \Sigma \times \Sigma$



(Lecture #5)

to

$\llbracket c \rrbracket_{\text{ok}} \subseteq \Sigma \times \Sigma$

$\llbracket c \rrbracket_{\text{er}} \subseteq \Sigma \times \Sigma$



(Lecture #6)

to

(this Lecture)

$\llbracket c \rrbracket \subseteq \Sigma_{\epsilon} \times \Sigma_{\epsilon}$

$\Sigma_{\epsilon} \triangleq \{\text{ok}, \text{er}\} \times \Sigma$

$\llbracket c \rrbracket \triangleq \{(\text{ok} : \sigma, \epsilon : \delta) \mid (\sigma, \delta) \in \llbracket c \rrbracket_{\epsilon}\} \\ \cup \{(\text{er} : \sigma, \text{er} : \sigma) \mid \sigma \in \Sigma\}$

Minimal set of rules

$$\begin{array}{c}
 \frac{}{[P] \ e \ [\llbracket e \rrbracket P]} \text{[atom]} \qquad \frac{[P] \ c_1 \ [R] \quad [R] \ c_2 \ [Q]}{[P] \ c_1; c_2 \ [Q]} \text{[seq]} \\
 \\
 \frac{\forall i \in \{1,2\} \ [P] \ c_i \ [Q_i]}{[P] \ c_1 + c_2 \ [Q_1 \cup Q_2]} \text{[choice]} \qquad \frac{\forall n \geq 0. \ [P_n] \ c \ [P_{n+1}]}{[P_0] \ c^\star \ [\exists k. P_k]} \text{[iter]} \\
 \\
 \frac{P' \Rightarrow P \quad [P'] \ c \ [Q'] \quad Q \Rightarrow Q'}{[P] \ c \ [Q]} \text{[cons]}
 \end{array}$$

Auxiliary rules

$$\frac{[P_1] \text{ } c \text{ } [Q_1] \quad [P_2] \text{ } c \text{ } [Q_2]}{[P_1 \vee P_2] \text{ } c \text{ } [Q_1 \vee Q_2]} \text{ [disj]}$$

$$\frac{}{[P] \text{ } c^\star [P]} \text{ [iter0]}$$

$$\frac{[P] \text{ } c^\star; c \text{ } [Q]}{[P] \text{ } c^\star [Q]} \text{ [unroll]}$$

$$\frac{[P] \text{ } c \text{ } [Q]}{[P \wedge R] \text{ } c \text{ } [Q \wedge R]} \text{ [frame]}$$

assigned variables in c
are disjoint from
free variables in R

$$\frac{P' \Rightarrow P \quad [P'] \text{ } c \text{ } [Q]}{[P] \text{ } c \text{ } [Q]} \text{ [weak]}$$

$$\frac{[P] \text{ } c \text{ } [Q'] \quad Q \Rightarrow Q'}{[P] \text{ } c \text{ } [Q]} \text{ [stren]}$$

Soundness

Th. Any derivable IL triple is valid

Proof. By induction on the derivation tree

Proof (sketch).

It suffices to show the local soundness of each rule.

For each axiom: it suffices to show it is valid.

For each inference rule: we assume that the premises are valid and we show that the conclusion is valid as well.

Soundness

Th. Any derivable IL triple is valid

Proof. Sample case [seq]
$$\frac{[P] c_1 [R] \quad [R] c_2 [Q]}{[P] c_1; c_2 [Q]} \text{ [seq]}$$

We assume the premises valid and show the conclusion is valid as well.

From $\llbracket c_1 \rrbracket P \supseteq R$ and $\llbracket c_2 \rrbracket R \supseteq Q$

we can conclude

$$\llbracket c_1; c_2 \rrbracket P = \llbracket c_2 \rrbracket (\llbracket c_1 \rrbracket P) \supseteq \llbracket c_2 \rrbracket R \supseteq Q$$

Soundness

Th. Any derivable IL triple is valid

Proof. Sample case [choice]
$$\frac{\forall i \in \{1,2\} \quad [P] c_i [Q_i]}{[P] c_1 + c_2 [Q_1 \cup Q_2]} \text{ [choice]}$$

We assume the premises valid and show the conclusion is valid as well.

From $\llbracket c_1 \rrbracket P \supseteq Q_1$ and $\llbracket c_2 \rrbracket P \supseteq Q_2$

we can conclude

$$\llbracket c_1 + c_2 \rrbracket P = \llbracket c_1 \rrbracket P \cup \llbracket c_2 \rrbracket P \supseteq Q_1 \cup Q_2$$

Soundness

Th. Any derivable IL triple is valid

Proof. Sample case [unroll]

$$\frac{[P] \ c^{\star}; c \ [Q]}{[P] \ c^{\star} \ [Q]} \text{ [unroll]}$$

We assume the premise valid and show the conclusion is valid as well. From $\llbracket c^{\star}; c \rrbracket P \supseteq Q$ we can conclude

$$\llbracket c^{\star} \rrbracket P = \bigcup_{k=0}^{\infty} \llbracket c^k \rrbracket P \supseteq \bigcup_{k=1}^{\infty} \llbracket c^k \rrbracket P = \llbracket c \rrbracket \bigcup_{k=0}^{\infty} \llbracket c^k \rrbracket P \supseteq \llbracket c^{\star}; c \rrbracket P \supseteq Q$$

(Relative) Completeness

independent of all but a finite
number of variables

involving **finitely-supported** predicates

Th. Any valid IL triple can be derived, provided we have an oracle to decide implications

Proof sketch. (Assuming an oracle to decide implications.)

Solution: we go semantic!

P and Q are sets of states, implications is subset inclusion, no issues with finiteness.

(Relative) Completeness

Proof sketch ctd.

First, we show that $[P] \ c \ \llbracket c \rrbracket P$ is derivable for any c and P .
(We do so by *induction* on the structure of c .)

Since any valid triple $[P] \ c \ [Q]$ satisfies $\llbracket c \rrbracket P \supseteq Q$,
we derive it using the rule of consequence

$$\frac{\frac{}{[P] \ c \ \llbracket c \rrbracket P} \quad \llbracket c \rrbracket P \supseteq Q}{[P] \ c \ [Q]}$$

(Relative) Completeness

Proof sketch ctd. ($[P] \ c \ \llbracket c \rrbracket P$ is derivable)

Roughly, by structural induction on the command c .

Atomic commands: $[atom]$

Choice and sequence: $by \text{ inductive hyp. } + [choice] / [seq]$

(Relative) Completeness: Kleene star

- $[P] c^\star [\text{ok} : \llbracket c^\star \rrbracket \text{ok} P]$

There exists a finite number of iterations of c that, starting from a state in P , reaches a state satisfying $\llbracket c^\star \rrbracket \text{ok} P$ (shortly written $\llbracket c^\star \rrbracket P$)

- inductive hyp. $[P] c \llbracket c \rrbracket P$
- $P_n \triangleq \llbracket c^n \rrbracket P$ and $P_0 \triangleq P$ we instantiate [iter] as follows

$$\frac{\forall n \geq 0. [P_n] c [P_{n+1}]}{[P_0] c^\star [\exists k. P_k]} \text{ [iter]}$$

by applying the inductive hypothesis we know that

$$\llbracket c^n \rrbracket P c \llbracket c \rrbracket (\llbracket c^n \rrbracket P) \text{ but } \llbracket c \rrbracket (\llbracket c^n \rrbracket P) \equiv \llbracket c^n; c \rrbracket P \equiv \llbracket c^{n+1} \rrbracket P$$

by applying applying [iter] we can conclude $[P] c^\star [\exists k. \llbracket c^k \rrbracket P] \equiv \llbracket c^\star \rrbracket P$

(Relative) Completeness: Kleene star

- $[P] c^\star [er : Q_{er}]$

There exists a finite number of iterations of c that, starting from a state in P , reaches a state satisfying $[er : Q_{er}]$

By inductive hyp. $\forall R, [R] c [er : Q_{er}]$ and by previous result we know that $[P] c^\star [\exists k. [[c^k]]P]$. By instantiating $R = \exists k. [[c^k]]P$ we have

$$\frac{[P] c^\star [\exists k. [[c^k]]P] \quad [\exists k. [[c^k]]P] c [er : Q_{er}]}{[P] c^\star; c [er : Q_{er}]} \quad \text{by } \frac{[P] c^\star; c [Q]}{[P] c^\star [Q]} [\text{unroll}]$$

we conclude $[P] c^\star [er : Q_{er}]$

Exercises on IL and HL

Mixed triples 1

Is this “mixed” HL+IL inference rule sound ?

$$\frac{\{P \wedge b\} c \{P\}}{[P] \text{ while } b \text{ do } c [\text{ok} : P \wedge \neg b]}$$

Hint: recall sample rules seen previously for IL

$$\frac{[P_0] \text{ while } b \text{ do } c [P_0 \wedge \neg b]}{\quad} \quad \frac{[P_0 \wedge b] c [P_1]}{\quad} \quad \frac{[P_0] \text{ while } b \text{ do } c [P_0 \wedge \neg b] \quad [P_0] \text{ while } b \text{ do } c [(P_0 \vee P_1) \wedge \neg b]}{\quad}$$

Mixed triples 1

Is this “mixed” HL+IL inference rule sound ?

$$\frac{\{P \wedge b\} c \{P\}}{[P] \text{ while } b \text{ do } c [\text{ok} : P \wedge \neg b]}$$

This rule is valid, because the conclusion is always valid

$$\frac{\frac{[P] (b?; c)^* [P]}{[P] (b?; c)^* (\neg b)? [P \wedge \neg b]} [\text{iter0}]}{[P] (b?; c)^* (\neg b)? [P \wedge \neg b]} [\text{seq}] \quad [\text{assume}]$$

Mixed triples 2

Is this “mixed” HL+IL inference rule sound ?

$$\frac{[P \wedge b] \ c \ [\text{ok} : P]}{\{P\} \ \text{while } b \ \text{do } c \ \{P \wedge \neg b\}}$$

Hint: take, e.g., while $(x > 0)$ do $x := x - 1$

$$\frac{[P \wedge (x > 0)] \ x := x - 1 \ [\text{ok} : P]}{\{P\} \ \text{while } (x > 0) \ \text{do } x := x - 1 \ \{P \wedge (x \leq 0)\}}$$

Mixed triples 2

Is this “mixed” HL+IL inference rule sound ?

$$\frac{[P \wedge b] \ c \ [\text{ok} : P]}{\{P\} \ \text{while } b \ \text{do } c \ \{P \wedge \neg b\}}$$

No. This rule is not sound.

Take, $P = b \triangleq (x > 0)$, $c \triangleq x := x - 1$, then the IL triple

$[x > 0] \ \text{while } x > 0 \ \text{do } x := x - 1 \ [\text{ok} : x > 0]$ is valid,

but the HL triple $\{x > 0\} \ \text{while } x > 0 \ \text{do } x := x - 1 \ \{false\}$

is not valid, because $\llbracket \text{while } x > 0 \ \text{do } x := x - 1 \rrbracket P \not\subseteq \emptyset$

Backward Analysis

Regular commands

regular
command

atomic
command

choice

$c ::=$

e

|

$c_1; c_2$

|

$c_1 + c_2$

|

$c^\star$

Kleene
star

$e ::= \text{skip} \mid x := a \mid b? \mid \dots$

Syntactic sugar

$\text{if } b \text{ then } c_1 \text{ else } c_2 \triangleq (b?; c_1) + (\neg b?; c_2)$

$\text{while } b \text{ do } c \triangleq (b?; c)^\star; \neg b?$

Backward analysis

Forward Analysis

```
int Simple (bool b)
{ int z;
  if (b)
    z := 12;  $z \in [12, 12]$ 
  else
    z := -12;  $z \in [-12, -12]$ 
     $z \in [-12, 12]$ 
  return 1/z;
}
```

interval analysis

Possible division by 0

Backward Analysis

```
int Simple (bool b)
{ int z;
  if (b) ✓
    ✓ z := 12;  $z \neq 0$ 
  else
    ✓ z := -12;  $z \neq 0$ 
  return 1/z;  $z \neq 0$ 
}
```

Backward semantics

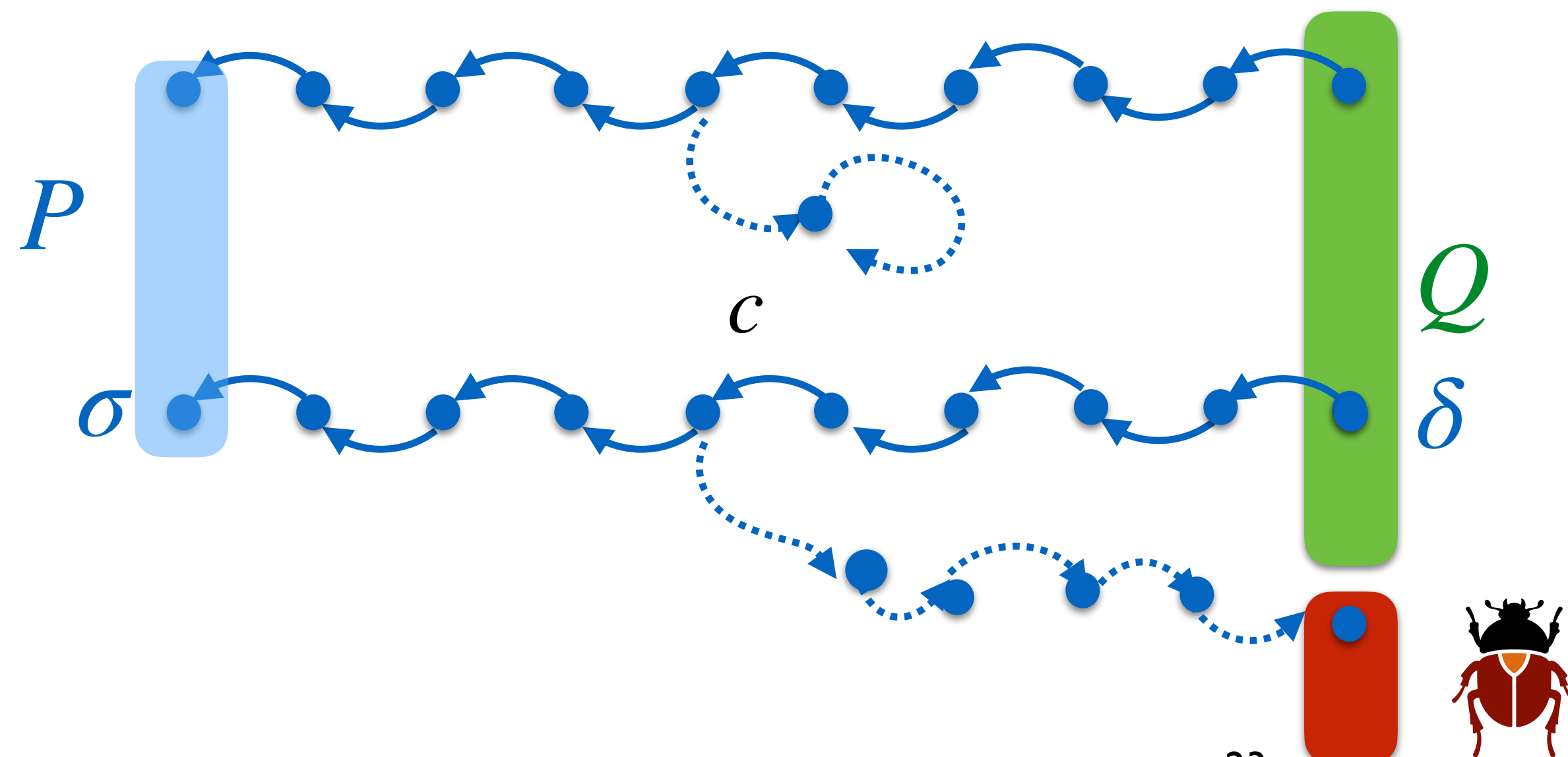
Different from WLP!

$$\llbracket c \rrbracket^{\text{op}} \delta \triangleq \{ \sigma \mid \delta \in \llbracket c \rrbracket \sigma \} \quad \sigma \in \llbracket c \rrbracket^{\text{op}} \delta \Leftrightarrow \delta \in \llbracket c \rrbracket \sigma$$

Hoare called it *weakest possible precondition*

As before we can extend it to sets

$$\llbracket c \rrbracket^{\text{op}} Q = \bigcup_{\delta \in Q} \llbracket c \rrbracket^{\text{op}} \delta$$



Example

Let us focus on natural numbers

$c \triangleq$
Divisor_of(x) {
 ...
 s := nondet[2..x/2];
 if (x % s == 0)
 skip
 else
 while true do skip
}

$$wlp(c, Q) \triangleq \{\sigma \mid \llbracket c \rrbracket \{\sigma\} \subseteq Q\}$$

$$wlp(c, [s = 5, x = 17]) = ??? \text{ (} x \text{ prime, 0, 1)}$$

$$wlp(c, [s = 5, x = 15]) = ??? \text{ (} x \text{ prime, 0, 1)}$$

$$\llbracket c \rrbracket^{op} Q \triangleq \{\sigma \mid \sigma' \in \llbracket c \rrbracket \sigma, \sigma' \in Q\}$$

$$\llbracket c \rrbracket^{op} [s = 5, x = 17] = ??? \emptyset$$

$$\llbracket c \rrbracket^{op} [s = 5, x = 15] = ??? \text{ (} x = 15)$$

Unexpected result?

Behind any programming activity there is an expectation

What the program **should do**
Sort an array, find the maximum...

What the program **should not do**
Read private data, divide by zero...
Loop forever, irresponsive user interface...

The expectation is the program **specification**

What is programming about?

Specification

What my code should do and should not do

Hacking

Write the code

Debugging / Verification

“Prove” my code follows its specification

Specification

Can be informal

No hang, no infinite loops, no crashes, no wrong output...

Can be formal

Simplest: Test cases

What I expect in some finite cases

More complex: formal specification language

Debugging / Verification

Runtime analysis

Run the program, observe the behavior for some specific runs
Check if the behavior violates the specification

Static analysis / verification

Do not run the program, observe the properties for all runs
Check if the behavior meets the specification

Example: Abs

Which is the specification for this code?

```
public int32 Abs(int32 x)
{
    if (x < 0)
        return -x;
    else
        return x;
}
```

Reminder:

$$\text{int32} = [-2^{31}, 2^{31} - 1]$$
$$-(-2^{31}) = -2^{31}$$

to return a non-negative value?
equal to the absolute value of x ?
does it work?
under which conditions?

Overview

Problem: Automatic inference of preconditions

Which preconditions?

Sufficient Precondition: if it holds, the code is correct

Sufficient Precondition: if it does not hold... we don't know

Necessary Precondition: if it **does not** hold, the code is not correct

Necessary Precondition: if it **does not** hold, something bad can happen

Sufficient preconditions

```
int Example1(int x,  
             object[] a)  
{  
    if (x >= 0)  
    {  
        return a.length;  
    }  
    return -1;  
}
```

Sufficient precondition: `(a != null)`

Too strong for the caller

no runtime errors also when
`(x < 0) and (a == null)`

Users of verification tools complained about it
“wrong preconditions”

Sufficient Precondition: if it holds, no error is raised

Necessary preconditions

```
int Example2 (object[ ] a)
{ for (int i=0; i<=a.length; i++)
  {
    a[i]=f(a[i]);
    if ( nondet() )
      return;
  }
}
```

Sufficient precondition: `false`

The function may fail
So eliminate all runs!

Necessary precondition: `(a != null &
0 < a.length)`

If `(0==a.length)` then it will always fail!

Sufficient Precondition: if it holds, no error is raised

Necessary Precondition: if it does not hold, something bad can happen

Necessary conditions (NC)

VMCAI 2013

Automatic Inference of Necessary Preconditions

Patrick Cousot¹, Radhia Cousot², Manuel Fähndrich³, and Francesco Logozzo³

¹ NYU, ENS, CNRS, INRIA

pcousot@cims.nyu.edu

² CNRS, ENS, INRIA

rcousot@ens.fr

³ Microsoft Research

{maf, logozzo}@microsoft.com

Abstract. We consider the problem of *automatic* precondition inference. We argue that the common notion of *sufficient* precondition inference (*i.e.*, under which precondition is the program correct?) imposes too large a burden on callers, and hence it is unfit for automatic program analysis. Therefore, we define the problem of *necessary* precondition inference (*i.e.*, under which precondition, if violated, will the program *always* be incorrect?). We designed and implemented several new abstract interpretation-based analyses to infer atomic, disjunctive, universally and existentially quantified necessary preconditions.

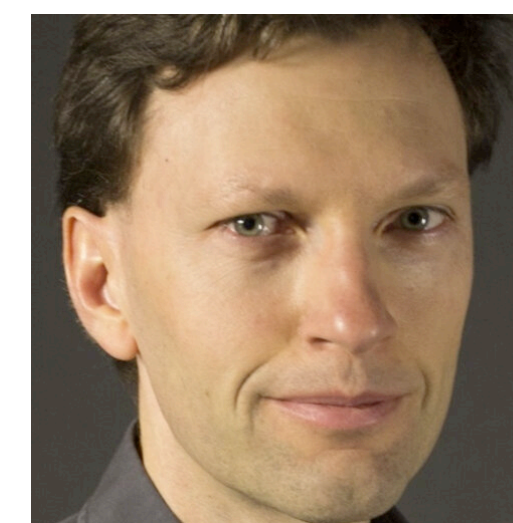
We experimentally validated the analyses on large scale industrial code. For unannotated code, the inference algorithms find necessary preconditions for almost 64% of methods which contained warnings. In 27% of these cases the inferred preconditions were also *sufficient*, meaning all warnings within the method body disappeared. For annotated code, the inference algorithms find necessary preconditions for over 68% of methods with warnings. In almost 50% of these cases the preconditions were also sufficient. Overall, the precision improvement obtained by precondition inference (counted as the additional number of methods with no warnings) ranged between 9% and 21%.

1 Introduction

Design by Contract [28] is a programming methodology which systematically requires the programmer to provide the preconditions, postconditions and object invariants (collectively called contracts) at design time. Contracts allow automatic generation of documentation, amplify the testing process, and naturally enable assume/guarantee reasoning for divide and conquer static program analysis and verification. In the real world, relatively few methods have contracts that are sufficient to prove the method correct. Typically, the precondition of a method is weaker than necessary, resulting in unproven assertions within the method, but making it easier to prove the precondition at call-sites. Inference has been advocated as the holy grail to solve this problem.

In this paper we focus on the problem of computing *necessary preconditions* which are *inevitable checks* from within the method that are hoisted to the

“Under which **precondition**, if violated, will the program always be **incorrect**?”



Necessary preconditions

When **automatic inference** is considered,
necessary preconditions make much more sense

Sufficient preconditions impose too large a burden to callers

Necessary preconditions are easy to explain to users

Implemented in the contract checker verifier **Clousot** (Microsoft)

Precision improvements 9% to 21%

Extremely low false positive ratio

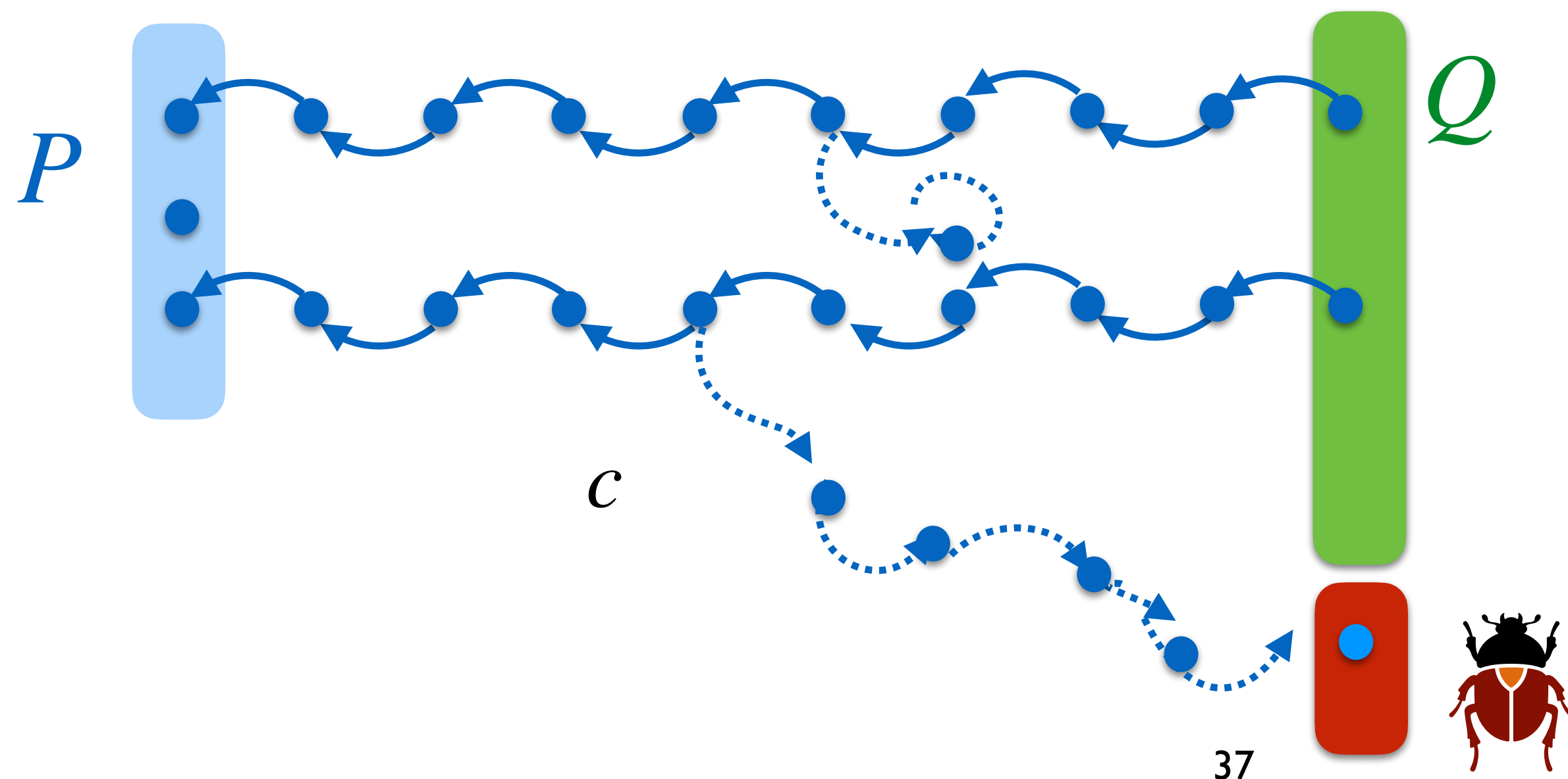
Necessary preconditions

The idea of NC is to prevent the invocation of the function with arguments that **will inevitably lead to some error**

Given Q the set of good final states (**the specification**), the NC triple

$$(P) \ c \ (Q)$$

means that any states σ that admits at least one non-erroneous execution of c is in P



$$\llbracket c \rrbracket^{op} Q \subseteq P$$

It is an over-approximation!

Necessary Conditions

$$(P) \text{ } c \text{ } (Q)$$

Program traces: $T = G \cup B \cup I$

G = good traces, terminating in a state in Q

B = bad traces, terminating outside Q in an assertion violation

assertions:

Language-induced: division by zero, null pointers, buffer overrun ...

User-supplied annotations: assertions, preconditions, postconditions, object invariants

I = infinite traces, non-termination

Notation: $G(\sigma)$ are the good traces starting from $\sigma, \dots$

Sufficient Conditions

$$wlp(c, Q)$$

If the state $\sigma \in wlp(c, Q)$, then no bad state is reachable starting from σ

The following holds :

$$\sigma \in wlp(c, Q) \implies B(\sigma) = \emptyset$$

So that it may exists some initial state σ such that

$$\sigma \notin wlp(c, Q) \wedge G(\sigma) \neq \emptyset$$

i.e., $\sigma \notin wlp(c, Q)$ but it does **not necessarily lead** to a **bad** state

Necessary Conditions

$$(P) \text{ } c \text{ } (Q)$$

If the state $\sigma \in P$, then σ cannot reach bad states only

Equivalently :

$$G(\sigma) \neq \emptyset \implies \sigma \in P \quad (\text{i.e., } \sigma \notin P \implies G(\sigma) = \emptyset)$$

$$\sigma \notin P \implies ((I(\sigma) = \emptyset) \implies B(\sigma) \neq \emptyset \wedge G(\sigma) = \emptyset)$$

i.e., if $\sigma \notin P$, either

a computation **diverge**, or reach only **bad states**

Comparison ignoring termination

Does σ belongs to the ...

Weakest sufficient preconditions

		$G(\sigma)$	
		$\emptyset$	$\neq \emptyset$
$B(\sigma)$	$\emptyset$	true	true
	$\neq \emptyset$	false	false

Strongest necessary preconditions

		$G(\sigma)$	
		$\emptyset$	$\neq \emptyset$
$B(\sigma)$	$\emptyset$	true	true
	$\neq \emptyset$	false	true

Compare approximation logics

	Forward	Backward
Over		
Under		

$$\{\text{HL}\} \quad \llbracket c \rrbracket P \subseteq Q$$

Compare approximation logics

	Forward	Backward
Over	$\{\text{HL}\} \quad \llbracket c \rrbracket P \subseteq Q$	
Under		

$$\llbracket \text{L} \rrbracket \quad \llbracket c \rrbracket P \supseteq Q$$

Compare approximation logics

	Forward	Backward
Over	$\{\text{HL}\} \quad \llbracket c \rrbracket P \subseteq Q$	
Under	$\llbracket \text{IL} \rrbracket \quad \llbracket c \rrbracket P \supseteq Q$	

$$(\text{NC}) \quad \llbracket c \rrbracket^{op} Q \subseteq P$$

Compare approximation logics

	Forward	Backward
Over	$\{\text{HL}\} \quad \llbracket c \rrbracket P \subseteq Q$	$(\text{NC}) \quad \llbracket c \rrbracket^{op} Q \subseteq P$
Under	$\llbracket \text{IL} \rrbracket \quad \llbracket c \rrbracket P \supseteq Q$	? Lecture #08

Which consequence rule?

$$(NC) \quad \llbracket c \rrbracket^{op} Q \subseteq P$$

There's not a proof system but...

{HL}

[IL]

$$\frac{P \Rightarrow P' \quad \{P'\} c \{Q'\} \quad Q' \Rightarrow Q}{\{P\} c \{Q\}} \quad \text{???} \quad \frac{P' \Rightarrow P \quad [P'] c [Q'] \quad Q \Rightarrow Q'}{[P] c [Q]}$$

Weaken the pre and strengthen the post!

$$(NC) \quad \frac{P' \Rightarrow P \quad (P') c (Q') \quad Q \Rightarrow Q'}{(P) c (Q)}$$

HL vs NC: weakest/strongest pre and post

Given P, we look for
the **strongest** Q

$$\{\text{HL}\} \quad \llbracket c \rrbracket P \subseteq Q$$

Given Q, we look for
the **weakest** P

Given Q, we look for
the **strongest** P

$$(\text{NC}) \quad \llbracket c \rrbracket^{op} Q \subseteq P$$

Given P, we look for
the **weakest** Q

HL vs NC: relation

$$\{P\}c\{Q\} \iff (\neg P)c(\neg Q)$$

that means

$$[[c]]P \subseteq Q \iff [[c]]^{op}(\neg Q) \subseteq \neg P$$

The proof

We prove one implication,
the other is similar

$\llbracket c \rrbracket P \subseteq Q \implies \llbracket c \rrbracket^{op}(\neg Q) \subseteq (\neg P)$ Let us assume $\llbracket c \rrbracket P \subseteq Q$

We want to prove $\llbracket c \rrbracket^{op}(\neg Q) \subseteq (\neg P)$

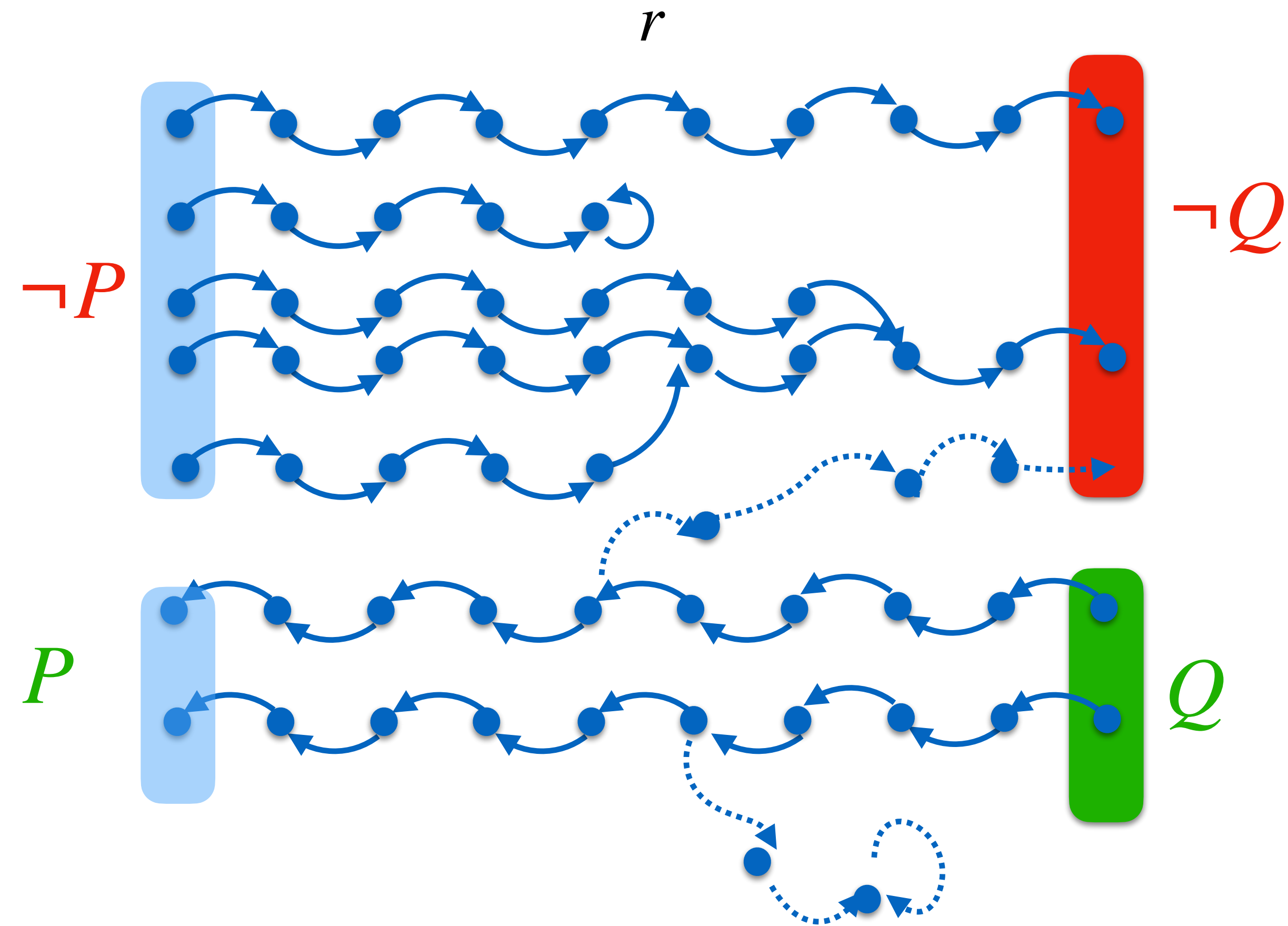
Let us take $\sigma \in \llbracket c \rrbracket^{op}(\neg Q)$ it must exist $\delta \in (\neg Q)$ such that $\sigma \in \llbracket c \rrbracket^{op} \delta$
i.e., such that $\delta \in \llbracket c \rrbracket \sigma$

if $\sigma \in P$, then it would mean that $\delta \in \llbracket c \rrbracket \sigma \subseteq \llbracket c \rrbracket P \subseteq Q$

but this is not possible, because we know that $\delta \notin Q$

therefore, it must be the case that $\sigma \notin P$

HL vs NC



$$\{\neg P\} \ c \ \{\neg Q\} \iff (P) \ c \ (Q)$$

$$\llbracket c \rrbracket(\neg P) \subseteq (\neg Q) \iff \llbracket c \rrbracket^{op} Q \subseteq P$$

Exercises

WLP vs WPP

Let $c \triangleq (z := x) + (z := y)$
and let $Q \triangleq (z = 0)$

What is $wlp(c, Q)$? $(x = y = 0)$ $wlp(c, Q) \triangleq \{\sigma \mid \llbracket c \rrbracket \{\sigma\} \subseteq Q\}$

What is $\llbracket c \rrbracket^{op} Q$? $(x = 0 \vee y = 0)$ $\llbracket c \rrbracket^{op} Q \triangleq \{\sigma \mid \sigma' \in \llbracket c \rrbracket \sigma, \sigma' \in Q\}$

Validity

Recalling that both $\{\text{false}\} \ c \ \{Q\}$ and $\{P\} \ c \ \{\text{true}\}$ are valid HL triples (for any P , Q and c)
can we claim something about the validity of NC triples such as

$$(\text{false}) \ c \ (Q) \iff \{\text{true}\} \ c \ \{\neg Q\} \quad \times$$

$$(P) \ c \ (\text{true}) \iff \{\neg P\} \ c \ \{\text{false}\} \quad \times$$

$$(\text{true}) \ c \ (Q) \iff \{\text{false}\} \ c \ \{\neg Q\} \quad \checkmark$$

$$(P) \ c \ (\text{false}) \iff \{\neg P\} \ c \ \{\text{true}\} \quad \checkmark$$